Asset Framework Documentation

This documentation covers all the scripts / textures / materials used in the Demo Scene.
This will help you understand how it all works, so that you can do your own tweaks / modifications or additions to the existing framework if required.

The common framework.
It is important to note that I am trying to create a common standard/framework for all my assets with navigable interiors (and others). This means common scripts, materials, even meshes. **Therefore if you familiarize yourself with how I structure my scenes/assets in one package, things should be familiar for my other assets** (that use the same standard).

Some assets from this documentation will not appear in your purchased pack.
Each pack only includes the required assets for their particular demo implementation. More complex asset packs will include more of the below mentioned assets.

If you are interested to know which of my assets use the shared framework, or if you have any other questions, please send me an e-mail at **VattalusAssets@gmail.com**

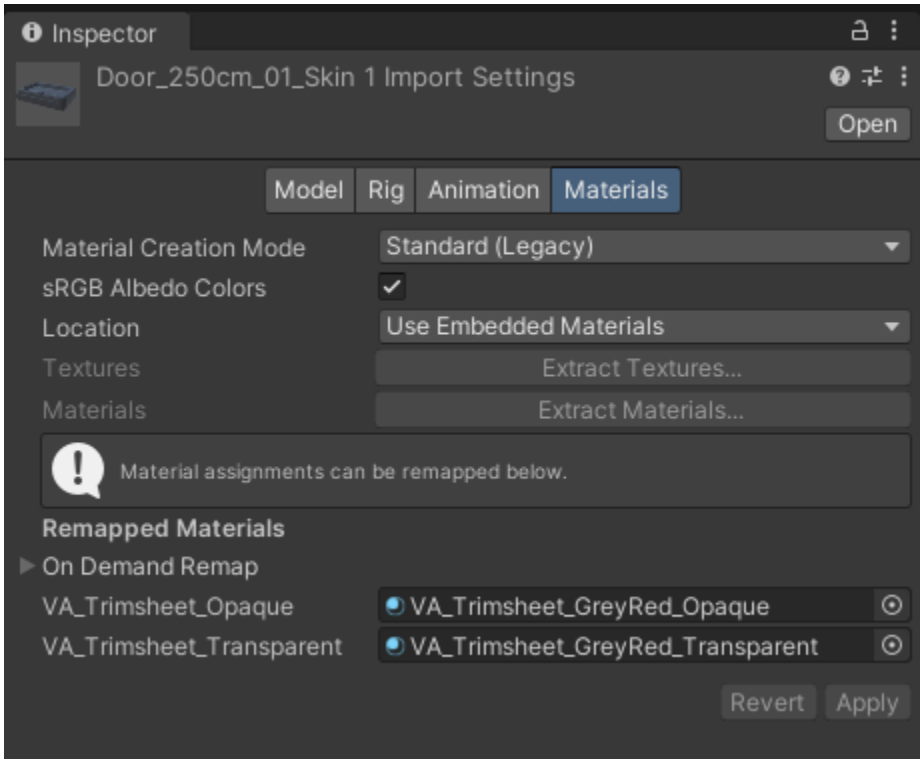
- **Materials and textures**

Main material and texture set
The meshes are textured using the trimsheet method. This means multiple meshes use the same material. Around 90% of the surfaces use the same 2 materials (one opaque, one transparent. Same textures but different shaders). Additional materials are used where the trimsheet is not enough (lights, glass, fabric etc).
The main advantage of using this method is: vastly reduces build size, memory usage and a reduction in drawcalls.

Location: VattalusAssets>Common>Materials>Trimsheet

Here you can find the base textures used for the trimsheet materials (in various formats, depending on your needs. My shaders are a modified version of the Unity Standard Shader in order to add a color tint function for 3 color zones, a merged Metal-Rough-AO map and a triplanar grunge effect). Multiple color variants are provided, but you can create your own by customizing the color tint values, or with the included .PSD file. You can customize the 3 different color zones (Primary color, Accent color and Neutral).

Changing trimsheet color variants
If you want to change the color variant or if something goes wrong, make sure to assign the 2 Trimsheet material versions to their respective slots. (Some meshes only use the Opaque material, some use both, some use additional material slots for the aforementioned additional materials like lights)

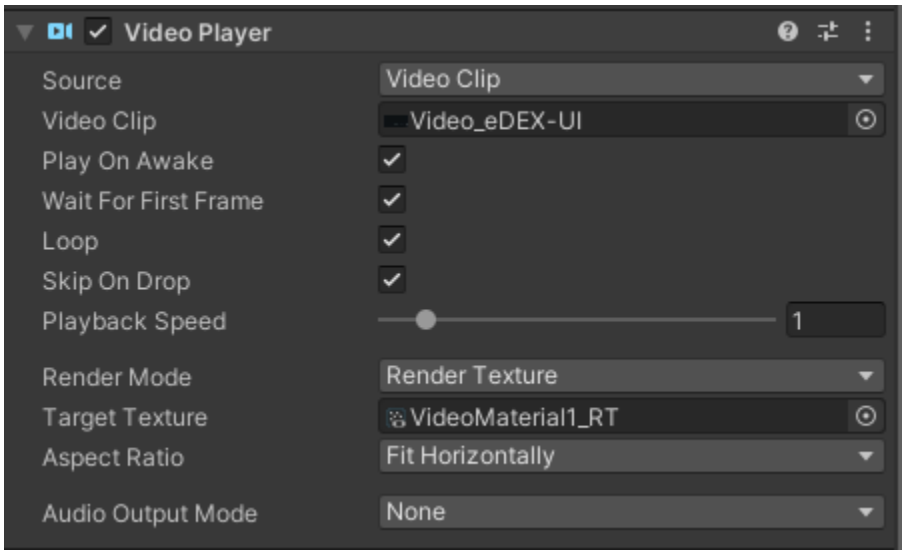


You can change the material color variant on individual sub-components of the mesh, like wall panels, inside the scene or prefabs.

Animated Lights Material
This is another material commonly used across many asset packs. It consists of a single texture map, a shader, and a material. It is used to texture various blinking lights of different colors and blinking patterns, all using the same material.

During texturing the desired light is mapped to a small area on the map, this will dictate its color and blinking pattern.

Video Materials
6 video materials are included. These are used decoratively on screens.
Each material consists of the video file, a render texture, and a material.
In order for it to work, this component needs to be in the scene (1 for each material).



Various other simple materials
An assortment of other example materials are included (such as emissive lights, fabric, different types of glass etc). These are used where the main trimsheet is not enough.

- **Scripts**

Several scripts are included in order to bring the demo scenes to life, usually to make interactions possible (open/close doors, sit in seats etc), and some are simply aesthetic (flickering lights). Below is a quick breakdown of the important scripts and their applications:

The **VattalusFirstPersonCamera** handles FPS movement and checks if the player is looking at an interactable object (a collider with an attached VattalusInteractible script component) and triggers an event with each interaction. (In my implementation the SceneController listens to these events, and passes them on to other scripts, such as the ShipController for when the player sits down in the pilot seat)

VattalusInteractable is commonly used on everything that the player interacts with (doors, drawers, switches, seats, beds etc). At the core, it is simply toggles between two states, with added functionality on each switch:

- Play a sound effect on activation/deactivation
- Play an animation (forward and in reverse) upon activation/deactivation
- Or, animate by transitioning between the initial position/rotation and a provided offset
- If “IsSeat” is enabled, upon interaction the FPS camera moves the to target anchor position, and a rotation angle restriction is applied.
- Toggle particle effects
- Trigger 4 distinct Event callbacks on activation/deactivation and when animation starts/ends.
- Can link to other interactables to propagate the interaction further. When “ConformLinked” is checked, propagation only occurs as to ensure the same state on all linked interactables. (Commonly used on doors with multiple moving panels. Each part has its own interactable script, because each script needs its own collider. When interacting with any part of a door, the interaction is propagated to all other parts to ensure all parts play the open animation).

The **VattalusSceneController** handles UI and player related logic. It has the following roles:

- References, updates and manages all the UI elements
- References and enables/disables the different camera views
- Passes along important events to other scripts such as the **VattalusSpaceshipController**

The **VattalusSpaceshipController** controls the ship-oriented behaviors:

- Sets key bindings that control the ship (movement, landing gear etc)
- Moves the Joystick and Throttle control inside the cockpit/bridge based on player input
- Deploys Ramp/Landing gear upon correct keypress
- Controls all the thrusters (VattalusThrusterController) based on ship movement key inputs
- Controls the different types of Engine/Thruster sounds (VattalusEngineSoundController)
- Controls the fake skybox rotation (VattalusFakeSkyboxMovement) when the ship is rotating in order to fake actual movement

VattalusInteriorEnvManager (formerly VattalusSpaceshipManager) mainly optimizes interior spaces (like spaceship interiors) by hiding rooms that are not visible to the player.
The optimization check is done:

- whenever a door is opened/closer
- whenever the player leaves the environment area (player is not found in any of the rooms)
- and when the camera mode is changed.

In order for all of this to work, the script needs to know whether the **VattalusFirstPersonCamera** is inside the environment, and if so, within which room it is.
It does this by checking all rooms (**VattalusRoomController**) among the children and only showing the one the player is inside in and the ones it is connected to either via an open door, or via a window.
(Rooms with permanent view between them have to have references to each other in the VattalusRoomController script).
In case the player is outside the ship (or when in orbit camera mode) only rooms marked with “VisibleFromExterior” are shown.

Theoretically multiple InteriorEnvManagers can work independently to optimize their individual room systems. It is important that InteriorEnvManagers scripts to not be parented to one another, as each of them has to have their own RoomController scripts childed to them.

VattalusRoomController controls aspects regarding individual rooms, for example show/hide meshes/lights, and control the volume of ambient sound effects (VattalusAudioController).
It also keeps track of the presence of **VattalusFirstPersonCamera** script entering/exiting the room. For this it needs a reference to one or more colliders that mark the bounds of the room with a VattalusPlayerDetector script added to said collider. Thus we can trigger certain events when the player enters/exits the room. OnPlayerEnteredRoom and OnPlayerLeftRoom events are exposed (For example we could easily implement a behavior where lights are toggled when the player enters/leaves the room)

VattalusDoorController can be viewed as an extension of the VattalusInteractable script, but with specific functionality for doors. It still uses VattalusInteractable components, but has the following extra functionality:

- References to 2 VattalusRoomController components that the door connects.
- Specific Event callbacks for then the door open/close starts/ends.
- Can automatically open/close when player enters/exits a bounds trigger collider (Requires a child object with a trigger collider, and a VattalusPlayerDetector script)

All other scripts are simple and self contained. If in doubt, refer to the annotations inside the code files. I tried to keep everything as well explained as possible.